Unified Line→Word→Grapheme Cascade for Bangla Handwriting

Executive Summary — BanglaWriting word-segmentation benchmark

27 September 2026

Bottom line

- We built `unified_cascade/`, one inference engine that runs our Swin-T Mask R-CNN line, word and grapheme models as a BN-DRISHTI-style cascade (page → lines → words), with the gains coming from better *inference*.
- On the 195 held-out BanglaWriting test pages, the naive cascade scores word F1 0.568. **Benchmark 1** (lines→words with inference tricks) raises this to **0.637**, and **Benchmark 2** (graphemes used to correct words) to **0.652**.
- With boxes calibrated to BanglaWriting's loose annotation style ($+x$), the same systems score 0.765 → **0.807** → **0.825**.
- BN-DRISHTI, scored with the same metric on the same pages, reaches 0.615. It produces 1.73 boxes per ground-truth word.
- Every threshold and classifier was fit on a separate 65-page dev split, and the test split was scored once. The test gains are consistent with the dev gains.
- **Not yet done:** the *single-weight* unified checkpoint is not trained. The results use the three existing stage checkpoints behind one engine.

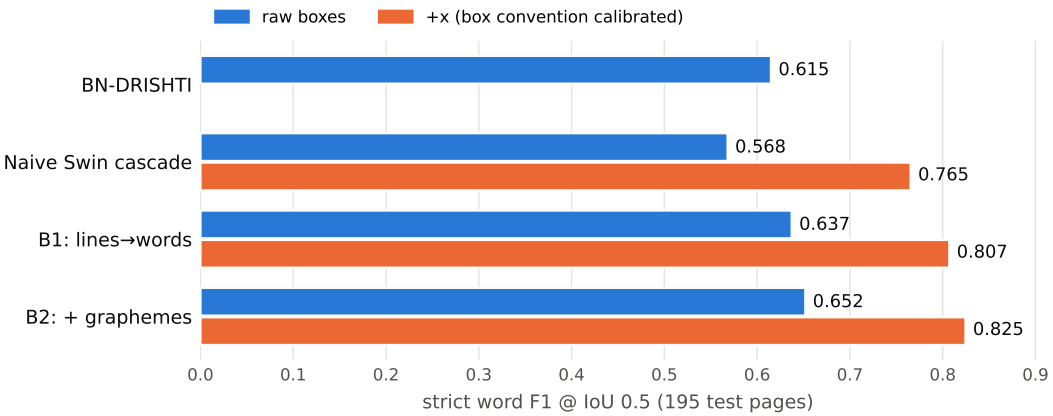


Figure 1: Held-out test word F1 (strict one-to-one matching, IoU ≥ 0.5). Blue: raw predicted boxes. Orange: boxes widened 5 % horizontally and 20 % vertically ("$+x$") to match BanglaWriting's hand-drawn margins. BN-DRISHTI's boxes are already loose (trained on BN-HTRd), so it has no $+x$ variant.

1. Problem and approach

We started with three independently trained Swin-T Mask R-CNN checkpoints (line, word, grapheme; 144M parameters in total), trained only on synthetic handwriting (syn-hw). BN-DRISHTI, the Bangla state of practice, runs a YOLOv5 line detector on the page and then a second YOLOv5 word detector on each line crop.

A literature review (Unified Detector, Hi-SAM, LayoutFormer, CRAFT, Cascade R-CNN/HTC, AutoFocus, SAHI) found no system that segments line ⊃ word ⊃ grapheme instances for a matra-connected script. We chose a **cascade with an intelligent inference layer** over a single one-shot pass, as requested. Each trick is behind a flag so that it can be ablated.

Evaluation protocol.

- **Data:** BanglaWriting, 260 real pages from 260 writers, never used for training.
- **Split:** dev = every 4th page (65 pages, 5,387 words); test = the remaining 195 pages (15,847 words).

- **Metrics:**
  - strict one-to-one F1 at IoU 0.5 and 0.75;
  - Wolf–Jolion split/merge F1;
  - COCO box AP;
  - per-error taxonomy: merged / split / loose / missed words, and spurious / partial predictions.

2. Benchmark 1 — lines → words (inference tricks)

Error analysis on the naive cascade showed that most losses came from *how crops are cut and merged*, not from model capacity. Neighbouring line crops overlap, so words are detected twice or leak across lines. We tested each fix on dev and kept only those that helped:

Trick	Effect on dev (F1@0.5)
Ownership masking (I1)	Ink closer to a neighbouring line's polygon is faded before word detection. Best single trick: 0.581 → 0.608 (hard, margin 0); the soft variant wins in combination.
Duplicate-aware merge (I2)	NMS on intersection-over-smaller-box across line crops. Mostly a precision gain (P 0.62 → 0.69).
Mask threshold 0.3, crop pad ×0.5	Fuller word masks and less neighbouring context: +0.03 and +0.013.
Line deskew (I3)	+0.02 alone after a bug fix, neutral once ownership masking is on, so left off.
Box convention ($+x$)	Not a detection change: it aligns ink-tight boxes with loose GT. Reported separately, never instead of raw.

Combined, these give `p1_best`: dev 0.581 → 0.660 and **test 0.568 → 0.637** (+0.069). Precision rises 0.598 → 0.708, split/merge F1 0.772 → 0.845, and runtime is 0.57 s/page.

3. Benchmark 2 — graphemes to predict words correctly

After Benchmark 1, the largest error class was **merged words**: one box covering two or more ground-truth words, 2,103 on test. We tried three designs before one worked.

1. **Rule-based split at grapheme gaps** ($> t \cdot H$). It cut merges by 44 %, but split errors rose ≈7× and F1 fell to 0.54–0.57. A pixel gap alone cannot tell intra-word from inter-word spacing.
2. **Learned gap arbitration with re-boxing.** Every gap between grapheme runs is classified from word-model, grapheme and ink features. Gap AUC reached 0.98, yet word F1 was only 0.58. Diagnosis: re-boxing *every* word from its graphemes alone lost 0.10 F1.
3. **Conservative edits (deployed).** The word model's boxes are kept, and graphemes only *split* a word where the classifier is confident ($P > 0.5$). A separate **punctuation** classifier adds or detaches marks (dari, comma, dash), which BanglaWriting labels as words. With no edits, the output is bit-identical to Benchmark 1, so every change is attributable to grapheme evidence.

What the graphemes contribute (page-grouped 5-fold cross-validation on dev):

- Gap AUC rises 0.932 with the word-model boundary alone → 0.965 with grapheme gap features → 0.970 with ink/matra-crossing features → 0.981 with an MLP.
- The matra (headline stroke), which joins graphemes *within* a word, is a useful, script-specific signal.

Test split (195 pages)	F1@0.5	F1@0.5 $+x$	SM-F1 $+x$	AP50 $+x$	merged	missed
Benchmark 1 (`p1_best`)	0.637	0.807	0.866	0.707	2103	1919
+ grapheme split (`g_learned`)	0.650	0.820	0.872	0.723	1613	1926
+ punctuation (`g_v3`, Benchmark 2)	**0.652**	**0.825**	**0.877**	**0.735**	**1478**	**1694**

On test, graphemes reduce merged-word errors by **30 %** (2,103 → 1,478) and missed words by 12 %. F1 improves +0.015 raw and +0.017 with $+x$, at a cost of about 1.05 s/page extra (1.62 vs 0.57 s/page).

4. Full results (held-out test)

System	F1@.5	P	R	F1@.75	SM-F1	AP50	s/page
BN-DRISHTI (YOLOv5 line→word)	0.615	0.485	0.841	0.219	0.674	0.582	—
Naive Swin cascade	0.568	0.598	0.541	0.058	0.772	0.356	0.47
Naive Swin cascade $+x$	0.765	0.806	0.729	0.458	0.824	0.665	0.47
Benchmark 1	0.637	0.708	0.580	0.099	0.845	0.436	0.57
Benchmark 1 $+x$	0.807	0.897	0.734	0.495	0.866	0.707	0.57
Benchmark 2	0.652	0.695	0.614	0.100	0.838	0.459	1.62
Benchmark 2 $+x$	**0.825**	0.879	**0.777**	0.495	**0.877**	**0.735**	1.62

How to read these numbers.

- Our raw F1@0.75 is low because our boxes hug the ink while BanglaWriting's boxes include margin. This is an annotation-convention gap, and $+x$ recovers it ($0.10 \rightarrow 0.50$).
- BN-DRISHTI's recall is high because it emits many overlapping boxes. Its precision is 0.485.

5. Quality assurance

Sonnet agents wrote the code, and independent Opus agents reviewed it before any number was trusted. The reviews found and fixed four bugs:

1. Deskew left the rotated crop uncut, which shrank text under the strip-height cap (deskew went from -0.036 to $+0.02$).
2. The ownership mask was misaligned by up to 3 px.
3. The punctuation step merged whole lines into one "mark".
4. A missing configuration would have crashed the test run.

The reviews also confirmed:

- there is no ground-truth use at inference;
- training and inference features match (live-vs-cached parity within 0.9 px);
- the out-of-fold results match in-sample ones (0.670 vs 0.670), and nested CV costs $\leq$0.001.

6. Remaining headroom and recommendations

Oracle analysis on dev ($+x$ F1, current 0.812–0.815) shows where the next gains are:

- **Punctuation** (oracle +0.05, captured +0.003). The classifier is weak (AUC 0.83; 651 positives out of 15.6k candidates). A dedicated small-mark detector trained on synthetic punctuation is the top recommendation.
- **Box geometry of split words** (oracle +0.05). Correct splits with GT-quality boxes would reach 0.905, against 0.851 with our current box geometry.
- **Split candidates.** 104 merged words have no grapheme gap at the true boundary, so split recall is capped by candidate generation.
- **Single-weight model.** Train `composite/` (shared backbone, three heads, -42 % parameters) with cascade-aware sampling. An estimated $\sim$1 GPU-week; the engine is model-agnostic and will run on it unchanged.
- **Speed.** The grapheme pass triples runtime. It could run only on words the classifier flags as likely merges, since most words need no grapheme evidence.

Artifacts

- **Code:** `unified_cascade/`. `engine.py` is the cascade and tricks; `gaps.py`, `v3.py` and `punct.py` are the grapheme and punctuation editors; `bench.py` is the benchmark.
- **Every statistic:** `unified_cascade/results/all_stats.json` (all 11 runs, per-page counts, CV/oracle tables). Regenerate it with `python -m unified_cascade.collect_stats`.
- **Narrative log:** `unified_cascade/RESULTS.md`.
- **Plan and literature:** `docs/cascade_clever_inference_plan.md`.